

Hybrid Zeroth-Order Fine-Tuning of Quantized LLMs via Adaptive Bin Shifting & Scale Optimization

A novel method for memory-efficient fine-tuning that directly optimizes discrete 4-bit weights without backpropagation.

The “Discrete Gradient” Barrier Prevents Efficient Fine-Tuning

The Constraint

Fine-tuning massive Large Language Models (LLMs) requires immense VRAM, making it inaccessible on consumer-grade hardware.

The Proposed Solution

Operate directly on memory-efficient 4-bit Quantized Weights.

The Fundamental Problem

4-bit weights are discrete step functions. Standard gradient-based optimizers fail because infinitesimal changes to inputs do not alter the integer output (e.g., $\text{Round}(4.1) = 4$ and $\text{Round}(4.101) = 4$).

The Consequence

This results in **Zero Gradients**, effectively freezing the model and making learning impossible.

Prior Art I: QZO Fails Because It Is Too Coarse

Concept

QZO (Quantization-Aware Zeroth-Order) fine-tunes a quantized model by learning a single global scale vector (s), which acts like a 'volume knob' for the entire layer, adjusting its dynamic range.

Mechanism

The scale s multiplies the original quantization step size Δ_{orig} .

Critical Failure

It is Too Coarse.

- Changing the scale affects every weight in the channel simultaneously.
- It cannot perform surgical updates required to fix specific facts or learn new knowledge (e.g., changing a single weight from integer value 4 to 5).
- Attempting to fix one weight with a global change inevitably distorts the entire layer's learned representation.

Prior Art II: QuZO Fails Because It Is Transient

Concept

QuZO (Quantized Zeroth-Order with Stochastic Rounding) introduces random noise to probabilistically "flip" bits during the training forward pass.

Mechanism

Uniform noise η is added to the weights before rounding. This creates a non-zero probability that a weight will round to a different integer, allowing a gradient signal to be estimated.

Critical Failure

It is Transient.

- The method relies on randomness to find gradients but has no structural mechanism to *store* the learned update.
- Once training is complete and the noise is removed for inference, the model 'snaps back' to its original state, losing the new information. Knowledge is not permanently encoded.

A Missing Link: The Need for Precise, Permanent, and Stable Updates

Synthesizing the Gap

- QZO provides **global stability** but lacks **local precision**.
- QuZO provides **local precision** but lacks a mechanism for **permanent storage**.

The Required Solution

A method is needed that unifies these concepts. It must:

1. Combine the global dynamic range tuning of scale learning.
2. Enable precise, bit-level changes for individual weights.
3. Incorporate a **learnable, structural component** to make these bit-level changes permanent.

Our Solution: Hybrid Adaptive Quantization via Coarse Scale + Fine Shift

Core Idea

We propose a unified framework that simultaneously optimizes two distinct aspects of the quantized layer using a Noise-Assisted Zeroth-Order method.

The 'Coarse + Fine' Mechanism

We decompose the fine-tuning task into two parallel operations:

Coarse Tuning (The QZO Component):

A learnable multiplier s stretches or shrinks the global quantization grid, optimizing the layer's overall dynamic range for stability.

Fine Tuning (The Adaptive Shift):

Learnable Low-Rank Adapters (A and B) are introduced to apply a targeted, directional 'push' to individual weights, moving them across rounding boundaries.

The Goal

If a weight needs to change from 4 to 5, the adapters learn a permanent positive shift factor to force it over the rounding threshold of 4.5.

Architectural Implementation: A Trainable Wrapper Around Frozen Weights

Our method modifies a standard linear layer by introducing a trainable wrapper around the frozen 4-bit integer weights (W_{int}). The architecture consists of three components operating in unison:

1. Frozen Core

W_{int} : The 4-bit integer weight matrix. It is **never** updated.

2. Coarse Controller (Global)

s : A learnable scale vector that adjusts the global dynamic range of the layer. Inherits the principle from QZO.

3. Fine Controller (Local)

A , B : Learnable Low-Rank Adapters that function as “**Bin Shifters.**” They learn a structured, directional force to adaptively push individual integer weights across quantization boundaries, replacing the purely stochastic nature of QuZO.

The Master Equation for the Trainable Forward Pass

For a frozen integer weight W_{int} , the effective weight W_{eff} used in the forward pass $Y = W_{\text{eff}}X$ is defined as:

$$W_{\text{eff}} = \text{Round}(W_{\text{int}} + \eta + \mathbf{1}_{d_{\text{out}} \times d_{\text{in}}} + A \times B) \odot (\Delta_{\text{orig}} \odot s)$$

Where $\eta \sim \text{U}[-0.5, 0.5]$ is random noise.

Component Breakdown

s (Coarse Scale Multiplier): Learns to expand or contract the quantization grid. This provides global stability.

$A \times B$ (The '**Bin Shifter**'): Learns a permanent, local directional push for each weight. This is the core novelty that stores the learned update.

η (The '**Stochastic Bridge**'): Fills the discrete gap between integers. It acts as a catalyst, allowing the Zeroth-Order optimizer to 'feel' the gradient signal even before $A \times B$ is strong enough to flip the bit on its own.

How the Stochastic Bridge Enables Permanent Adaptation

The interaction between noise and the adapters creates a path through the non-differentiable landscape. Consider a weight W_{int} with a value of 4:

1. Initialization

At the start, the adapter product $A \times B$ is near zero. The effective weight is $\text{Round}(4 + \eta)$.

2. Noise Injection

The noise η causes the value to flicker between 3.5 and 4.5. The model output fluctuates accordingly.

3. Optimization Signal

The ZO optimizer observes that when noise is positive (e.g., $\eta = +0.4$, pushing the value to 4.4), the training loss drops. It senses a '**desirable direction**.'

4. Adaptation

The optimizer updates A and B to create a permanent positive push, reinforcing this **desirable direction**. For example, $A \times B$ learns a value of +0.15.

5. The Result

The effective weight is now $\text{Round}(4 + 0.15 + \eta)$. It is centered at 4.15 and more frequently rounds up to 5.

Inference Mode: Learned Shifts Become Permanent Bit Flips

Once training is complete, the stochastic components are removed to ensure deterministic output.

The “Lock-In” Process

1. The random noise η is removed.
2. The adapters A and B and the scale s are frozen.
3. The forward pass becomes fully deterministic:

$$W_{\text{eff}} = \text{Round}(W_{\text{int}} + 1_{d_{\text{out}} \times d_{\text{in}}} + A \times B) \odot (\Delta_{\text{orig}} \odot s)$$

Why It Works

The optimizer’s goal during training was to make $A \times B$ large enough to cross the rounding threshold.

- **If the learned shift is sufficient:** $W_{\text{int}} + (A \times B) = 4 + 0.15 = 4.15$. $\text{Round}(4.15)$ is still **4**. The bit did not flip. The model remains stable.
- **If the learned shift is stronger:** $W_{\text{int}} + (A \times B) = 4 + 0.51 = 4.51$. $\text{Round}(4.51)$ becomes **5**. The bit flip is now **permanent and deterministic**.

Optimization via Simultaneous Perturbation Stochastic Approximation (SPSA)

Since the Round() function is non-differentiable, we estimate gradients for the trainable parameters $\Theta = \{\mathbf{A}, \mathbf{B}, \mathbf{s}\}$ using **ZO-SPSA**.

1. Perturbation Step

Generate a random Gaussian perturbation $\mathbf{Z}$ and evaluate the loss $\mathbf{L}$ at two points:

- Positive Pass: $L^+ = L(\text{Model}(\Theta + \beta\mathbf{Z}))$
- Negative Pass: $L^- = L(\text{Model}(\Theta - \beta\mathbf{Z}))$

**(Note: The forward pass noise η is regenerated independently for each evaluation)*

2. Gradient Estimation

Estimate the gradient $\mathbf{g}$ using the finite difference of the loss:

$$\mathbf{g} = \frac{L^+ - L^-}{2\beta} * \mathbf{Z}$$

This single scalar difference drives the update for all parameters.

3. Parameter Update

Apply the update using a standard optimizer like SGD:

$$\Theta_{t+1} = \Theta_t - \alpha_t \mathbf{g}_t$$

Memory Complexity Analysis Reveals Substantial VRAM Savings

The ZO approach eliminates the need to store gradients or optimizer states for the main parameters, drastically reducing peak memory usage.

Comparative Memory Footprint

Component	Standard Fine-Tuning (LoRA)	Proposed Hybrid ZO Method
Weight Storage	4-bit + 16-bit Adapter	4-bit + 16-bit Adapter
Activation Storage	$O(L \times \text{Batch} \times \text{Seq})$	Zero (Inference Mode only)
Gradient Storage	$O(\text{Parameters})$	Zero (Estimated on-the-fly)
Optimizer States	$O(\text{Parameters})$	Zero (Simple SGD-like Update)
Peak VRAM (7B Model)	~16 GB	~6 GB

Our Contribution: Structuralizing Stochastic Gradients into Adapters

The key innovation is the synthesis of existing concepts with a novel mechanism for storing learned information.

Component	Function	Origin / Inspiration
Scale s	Optimizes global dynamic range	QZO
Noise η	Enables gradient flow across discrete steps	QuZO
Adapters A, B	Stores the bit-flip decision permanently	Novel (Our Contribution)

The Synthesis: This method effectively 'structuralizes' the transient, stochastic gradients of QuZO into permanent low-rank adapters, solving the core deficiency of previous stochastic methods.

A Unified Method for Backpropagation-Free Fine-Tuning

This work introduces a complete framework for fine-tuning quantized models, built on three novel principles:

1. **Adaptive Bin Shifting**

We replace static rounding with a **dynamic, learnable rounding boundary** boundary controlled by low-rank adapters. This provides a mechanism for **precise, permanent weight updates**.

2. **Stochastic Gradient Bridge:** We utilize noise injection not merely for stochasticity, but as a mathematical necessity to tunnel gradient signals through discrete step functions in a Zeroth-Order optimization setting.

3. **Unified Coarse-Fine Optimization**

This is the first proposed method to simultaneously optimize a layer's quantization grid (via Scale) and the specific integer placement of its weights (via Shift) **without requiring backpropagation**.